QUANTUM HACKATHON SPRINT

Simulation of BB84 Quantum Key Distribution

Secure key exchange — with detectable eavesdropping.

TEAM

SUPER POSITION SQUAD

Katta Sai Ritesh
Voona Nikhil Sai
Tangella Sri Hema Charan Teja

CHALLENGE

Problem Statement 7
BB84 Quantum Key Distribution (RPI)

PROJECT PITCH

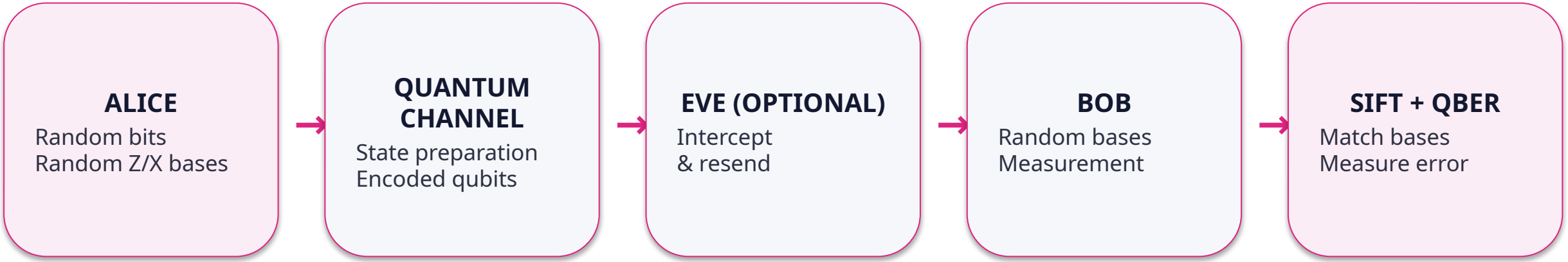
A practical BB84 simulator that creates a shared key, models intercept-resend, and uses QBER to make eavesdropping visible.

Problem & Motivation

PROBLEM STATEMENT

Can two users, Alice and Bob, securely establish a shared secret key using the BB84 Quantum Key Distribution protocol over an insecure quantum channel, while detecting a possible eavesdropper by calculating the Quantum Bit Error Rate (QBER)?

Proposed Solution



Eve OFF → low error → candidate key can be accepted

Eve ON → extra error → possible interception → reject

Every stage produces data that can be shown in a live demo.

Quantum Implementation

BB84 → QISKIT MAPPING

Z basis → computational basis
X basis → H gate before measurement
Bit encoding → |0>, |1> or H|0>, H|1>
Measurement → classical bit result
Eve → intermediate measurement + resend
Sifting → classical basis comparison
QBER → mismatch calculation

QISKIT-STYLE CORE

```
# Prepare / measure a BB84 qubit
if alice_basis == 1:
    qc.h(q)

# Bob chooses measurement basis
if bob_basis == 1:
    qc.h(q)

qc.measure(q, c)

# Classical sifting
if alice_basis == bob_basis:
    sifted_key.append(result)
```

CURRENT STATUS

The supplied Python layer abstracts measurement with random outcomes. The hackathon quantum stage replaces those outcomes with Qiskit circuits and validates the same logic on ideal/noisy simulators and, where available, IBM Quantum hardware.

Results & Verification

METRIC 01

SIFTED KEY LENGTH

Only positions where Alice and Bob used the same basis are retained.

Expected: roughly half of transmitted rounds.

METRIC 02

QBER

QBER = mismatched sifted bits / total sifted bits × 100

Expected: low in ideal no-Eve runs; higher with intercept-resend.

METRIC 03

KEY DECISION

Accept below the selected prototype threshold.

Reject when QBER is too high and flag possible eavesdropping.

FINAL EVIDENCE: generate QBER / counts from actual completed Qiskit runs.

No invented IBM hardware results are claimed in this deck.

Quantum Advantage

THE ADVANTAGE IS DETECTABLE DISTURBANCE — NOT SPEED.

CLASSICAL VIEW

Security often relies on the assumed computational difficulty of solving a mathematical problem.

BB84 VIEW

Security is linked to quantum-state behavior: measuring an unknown state can disturb it, making interception observable through errors.

Our demo makes that difference visible by running the same workflow with Eve OFF and Eve ON.

Future Scope

1 PYTHON PROTOTYPE
Protocol logic

2 QISKIT CIRCUITS
Real qubit preparation

3 NOISY SIMULATION
Channel + device effects

4 IBM QUANTUM
Hardware counts / statistics

NEXT FEATURES

Error correction • privacy amplification • finite-key analysis • richer channel-noise models • live Alice / Eve / Bob dashboard

Roadmap: Python → Qiskit → noisy model → IBM Quantum hardware

Implementation Code & Demo Logic

Exact Python prototype provided by the team - protocol simulation with Eve OFF / ON and QBER decision.

```
                    import random

def bb84_simulation(n=30, eve=False):

    # Alice generates random bits and bases
    A = [random.randint(0, 1) for _ in range(n)]
    Ab = [random.randint(0, 1) for _ in range(n)]

    # Bob chooses random bases
    Bb = [random.randint(0, 1) for _ in range(n)]

    B = []

    # Quantum transmission
    for i in range(n):

        if not eve:
            # No eavesdropper
            B.append(
                A[i] if Ab[i] == Bb[i]
```

```
                else random.randint(0, 1)
            )

        else:
            # Eve chooses a random basis
            eb = random.randint(0, 1)

            # Eve measures Alice's qubit
            e = (
                A[i]
                if eb == Ab[i]
                else random.randint(0, 1)
            )

            # Bob measures Eve's qubit
            B.append(
                e
                if Bb[i] == eb
                else random.randint(0, 1)
            )
```

```
    # Sifting
    # Alice and Bob keep bits where their bases match
    ak = []
    bk = []

    for i in range(n):

        if Ab[i] == Bb[i]:
            ak.append(A[i])
            bk.append(B[i])

    # QBER calculation
    errors = sum(a != b for a, b in zip(ak, bk))

    qber = errors / len(ak) * 100 if ak else 0

    # Decision
    threshold = 11

    if qber <= threshold:
        status = "KEY ACCEPTED"
    else:
        status = "KEY REJECTED - POSSIBLE EAVESDROPPING"

    return ak, bk, qber, status


# Without eavesdropper
print("WITHOUT EAVESDROPPER")
print(bb84_simulation(30, False))

print()

# With eavesdropper
print("WITH EAVESDROPPER")
print(bb84_simulation(30, True))
```